

IIVP2026 Group 14 Report

Keez Cuijpers^a, Leon Luczyn^a and Eira Gardner^a

^a*Department of Advanced Computing Sciences, Maastricht University, Maastricht, The Netherlands*

ARTICLE INFO

Keywords:

KEN3238

IIVP2026

Hindi-MNIST

Digit Recognition

ABSTRACT

This report has been submitted for the KEN3238 - Introduction to Image and Video Processing course in Department of Advanced Computing Sciences at Maastricht University. The report is a requirement of the IIVP-2026 challenge. The challenge has been created by Dr. Yusuf Can Semerci and Dr. Gaurvi Goyal (Semerci and Goyal, 2026).

1. Preprocessing

Before training the CNN, each input image was transformed into a standardized 32×32 grayscale representation. The purpose of this step was to reduce irrelevant variation in the raw images, such as differences in digit position, scale, background intensity, and stroke thickness, while preserving the overall glyph structure. Two preprocessing pipelines were evaluated. The first pipeline converted the image to grayscale, applied Otsu thresholding to separate foreground and background pixels, extracted a tight bounding box around the digit, centered the cropped digit on a square canvas, and resized the result to 32×32 pixels. The second pipeline followed a similar centering and resizing strategy, but used a fixed binarization threshold after normalization to $[0, 1]$ and applied a mild dilation operation to make thin strokes more continuous. These preprocessing choices were motivated by the visual characteristics of handwritten digits: the class-relevant information is mainly contained in the shape and relative arrangement of strokes, while the absolute location and background are less informative. Both variants produced very similar validation scores, with the Otsu-based pipeline reaching 0.9973404 and the dilation-based pipeline reaching 0.9946808. This small difference suggests that the CNN was robust to minor changes in binarization and stroke enhancement, as long as the digit was centered, normalized, and resized consistently. Therefore, preprocessing mainly served to standardize the input distribution rather than to introduce a major performance improvement. (Otsu, 1979)

2. Choice of Model

A convolutional neural network was selected because the input data consists of digit images with strong local spatial structure. In handwritten digit recognition, important visual features such as edges, curves, corners, loops, and stroke intersections usually appear in local neighbourhoods of pixels. A CNN is well suited to this type of data because convolutional filters learn local feature detectors and apply the same detector across the entire image. This weight sharing makes the model more parameter-efficient than a fully connected network and allows it to recognise similar stroke patterns regardless of their exact position in the image. This is particularly useful for handwritten digits, where the same digit class can vary in writing style, scale, thickness, and alignment.

The hierarchical structure of a CNN also matches the structure of the recognition task. Early convolutional layers can learn simple low-level features, such as oriented strokes or contrast changes, while deeper layers can combine these features into more complex digit-specific patterns. For example, combinations of curves and intersections may help distinguish visually similar classes. Since our preprocessing already centered and resized the digits, the CNN did not need to learn invariance to large translations, but it still benefited from local receptive fields and shared filters. For this reason, a CNN provided an appropriate balance between model complexity, computational efficiency, and suitability for image-based digit classification. (LeCun et al., 1998; Deng, 2012)

3. Model Design

Two CNN architectures were evaluated during model development. The first model, which was selected as the final architecture, was a compact DevanagariCNN operating on 32×32 grayscale inputs. It consisted of five convolutional layers arranged into three convolutional blocks, using a filter progression of 64, 128, 256 channels. Each convolution used a 3×3 kernel and was followed by batch normalization and a nonlinear activation. Max-pooling was applied after each block to gradually reduce the spatial resolution while increasing the semantic complexity of the learned features. Dropout2d was also applied after the convolutional blocks, and an additional dropout layer was used in the classifier to reduce overfitting.

The second model used a simpler CNN design with 64×64 grayscale inputs and three single convolutional layers with 32, 64, and 128 filters. Although the larger input resolution preserved more spatial detail, this model flattened the final feature maps directly before the dense classifier. As a result, the classifier received an $8 \times 8 \times 128 = 8192$ dimensional vector, producing a large fully connected layer and substantially increasing the number of trainable parameters. In contrast, the final DevanagariCNN used global average pooling, reducing the final feature maps to a 128-dimensional representation before classification. This made the selected model more parameter-efficient, with approximately 160K trainable parameters compared to approximately 660K in the second model.

A major architectural difference was the use of batch normalization and global average pooling in the selected model. Batch normalization was applied after every convolution, improving training stability and helping the network learn more consistent intermediate representations. Global average pooling reduced the dependence on large dense layers and encouraged the convolutional filters to act as class-relevant feature detectors. The selected model also

produced raw logits, leaving the softmax operation to the cross-entropy loss function, which is numerically more stable than applying an explicit softmax inside the model.

The second model achieved strong performance on most classes, correctly classifying digits 0–8 almost perfectly, with only one error among 15,300 examples. However, it failed completely on digit 9, reaching 0% accuracy for that class. Several preprocessing variants were tested to investigate whether this behaviour was caused by input formatting, binarization, centering, contrast inversion, or stroke thickness. These included normalization with contrast inversion, fixed-threshold binarization, tight cropping, centering, resizing, and mild dilation. None of these preprocessing changes resolved the failure on digit 9, suggesting that the issue was not primarily caused by preprocessing. Instead, the failure likely resulted from the second model’s design or training dynamics, such as its large flattened dense layer, lack of normalization, weaker regularization in the convolutional blocks, or unstable class-specific representation learning.

For these reasons, the first model was selected as the final architecture. It combined deeper convolutional feature extraction, batch normalization, multi-level dropout, and global average pooling, resulting in a more robust and parameter-efficient design. The comparison showed that increasing input resolution and parameter count did not necessarily improve reliability. A smaller but better-regularized CNN was more suitable for this task, especially because handwritten digit recognition depends on learning stable stroke-level and shape-level features rather than memorizing high-dimensional flattened representations. (Srivastava et al., 2014) (Lin et al., 2014)

4. Hyperparameter tuning

Selecting appropriate hyperparameters for deep learning models is a non-trivial task, as the large number of interdependent parameters makes exhaustive search computationally expensive, bordering on infeasible. To address this, Bayesian optimisation was employed using the Optuna framework with the Tree-structured Parzen Estimator (TPE) sampler. Unlike grid search or random search, Bayesian optimisation constructs a probabilistic surrogate model of the objective function, using previous trial results to guide the selection of promising configurations. This approach is substantially more sample-efficient, requiring fewer trials to converge on high performing regions of the search space. (Watanabe, 2025) (Nair, 2022) While it is true that random search requires fewer iterations and is the fastest method, it is highly dependent on the hyperparameter sets that are selected at random.

The hyperparameter search space was defined across three categories: training parameters, model architecture, and data augmentation. Training parameters included the learning rate, sampled log-uniformly over $[1e-4, 1e-2]$, weight decay over $[1e-6, 1e-2]$, batch size over $[16, 128]$, and maximum number of training epochs over $[20, 100]$. Architectural parameters included the convolutional channel configuration, dropout rates, and dense layer size. Augmentation parameters included rotation degrees, translation range, and scale range. To ensure validity, the scale minimum and maximum were sampled from non-overlapping intervals - $[0.7, 1.0]$ and $[1.0, 1.3]$ respectively - guaranteeing that $scale_min \leq scale_max$ across all trials. The loss function was selected categorically between standard cross-entropy and cross-entropy with label smoothing, with the label smoothing coefficient sampled conditionally only when the latter was chosen, avoiding wasted search budget on a parameter with no effect under standard cross-entropy.

Each trial trained a DevanagariCNN model from scratch using the sampled configuration, with early stopping applied with a patience of 10 epochs to prevent unnecessary computation on non-converging trials. Validation accuracy was used as the objective metric, which Optuna maximised across trials. A total of 400 trials were executed, with the first 10 consisting of random exploration before TPE-guided search began. The optimisation was run for approximately 12 hours. Of the 400 trials, 103 trials completed fully and were recorded. The remaining 297 trials were either pruned by Optuna’s MedianPruner - which terminates unpromising trials early based on intermediate validation accuracy relative to completed trials - or encountered training errors and were excluded from consideration. The optimisation results and parameter importance analysis are therefore based on the 103 fully completed trials.

It should be noted that running a large number of trials against the same validation set introduces the risk of implicit overfitting to that set.

4.1. Optimisation Results and Analysis

In Figure 1 the plot reveals that the best-performing configurations were identified relatively early in the search process, with the best-so-far curve plateauing at trial 41 and remaining stable for the remainder of the 400 trials. This suggests that TPE converged on high-performing regions of the search space efficiently, and that the majority of the computational budget was spent confirming rather than improving upon the best known configuration. The overall range

of the validation accuracy across all trials was narrow, spanning roughly 99.76% to 99.96%, which indicates that the model is relatively insensitive to hyperparameter variation within the defined search space.

The convergence analysis in Figure 2 reinforces this observations. The 10-trial moving average stabilises around 99.88% after approximately trial 20, with no meaningful upward trend observable in the latter half of the search. The continued variance in individual trial results throughout the run reflects the exploratory nature of TPE, which deliberately samples some configurations away from the known optimum to avoid premature convergence on local optima.

The hyperparameter importance analysis, computed using Optuna's built-in fANOVA-based importance estimator, provides a more rigorous quantification of each parameter's contribution to outcome variance, and can be seen in Figure 3. The results show that there are three parameters which are the most influential parameters, these are;

- Learning rate
- Max epochs
- Rotation degrees

Learning rate was identified as the most influential parameter by a substantial margin, with an importance score of 0.294, followed by max_epochs (0.190) and rotation_degrees (0.151). Batch size contributed a moderate importance of 0.089, while scale_min (0.066) and translation_range (0.061) showed similar modest influence. Scale_max (0.051) and dense_size(0.048) contributed marginally, and the remaining parameters - weight_decay (0.020), loss_name (0.010), dropout_rates (0.010), and conv_channels (0.009) - had negligible individual importance scores.

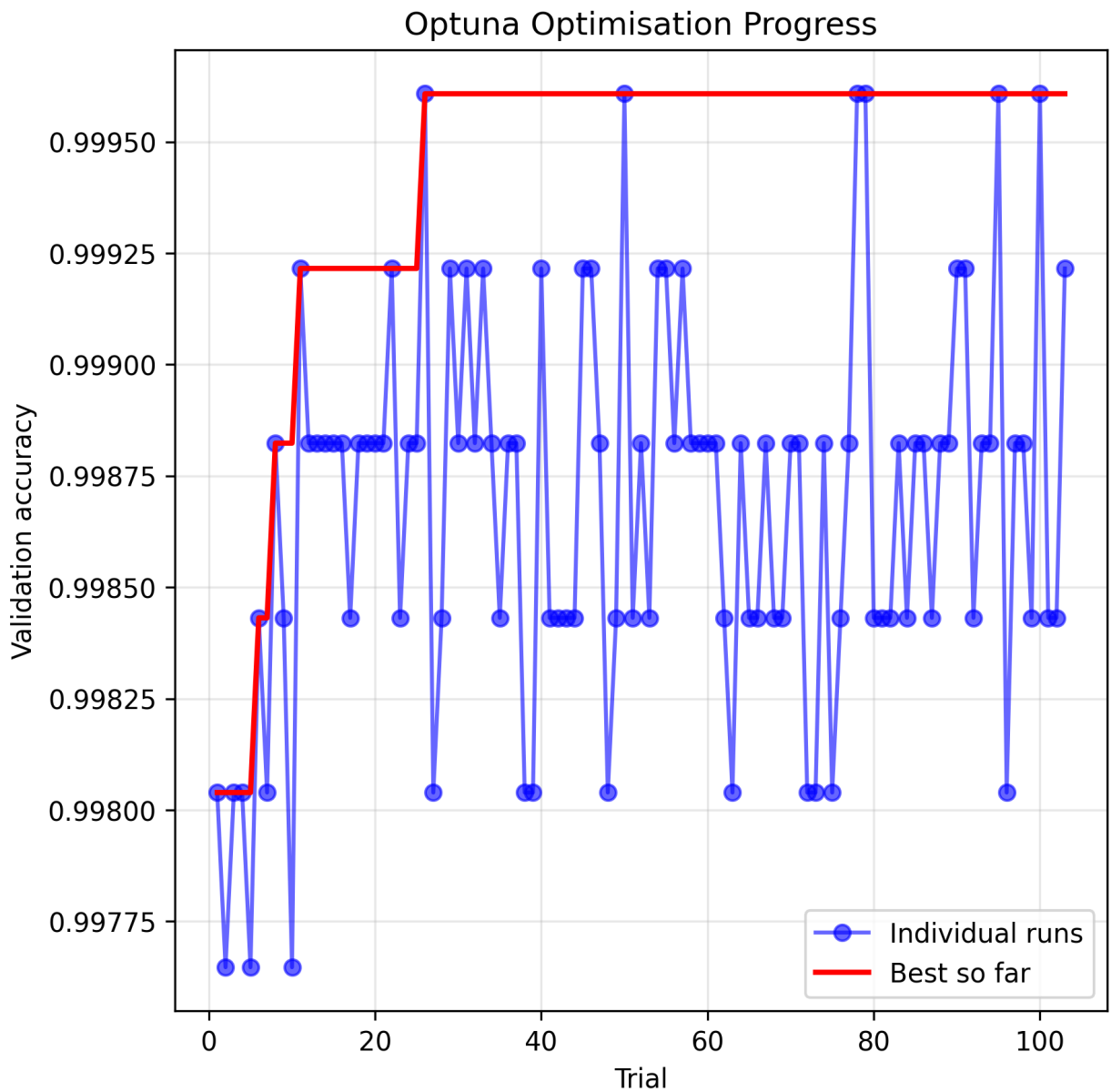
These findings carry several practical implications. The dominance of learning rate and max_epochs suggests that training dynamics were the primary driver of performance variation across trials, rather than architectural choices. The relatively low importance of conv_channels and dropout_rates is noteworthy given that these are often treated as critical design decisions. It suggest that within the ranges tested, all channel configurations were capable of reaching comparable accuracy levels on this dataset. The low importance of loss_name and weight_decay further supports the earlier observation that the model is robustly trainable across a range of regularisation settings. Conversely, the non-trivial importance of rotations_degrees highlights that augmentation strategy meaningfully influenced outcomes, consistent with the expectation that rotation invariance is relevant for handwritten character recognition.

5. Conclusion and Discussion

Overall, the final CNN-based pipeline performed very well while remaining relatively small and efficient. The comparison between the two architectures showed that a model with fewer trainable parameters can outperform a larger model when it uses a more suitable design, including batch normalization, dropout, and global average pooling. The preprocessing experiments also showed that the model was not highly sensitive to small changes in binarization, centering, or stroke enhancement, since the different preprocessing variants produced very similar scores. This suggests that the main performance gain came from the CNN architecture rather than from aggressive image preprocessing. Another practical advantage was that training was surprisingly fast and could be performed on a standard laptop, which made iterative experimentation feasible. However, the comparison also revealed an important limitation: the second model performed well on most classes but failed completely on digit 9, indicating that high overall accuracy can hide severe class-specific errors. For this reason, confusion matrices were used in addition to accuracy to evaluate the models more carefully. Example images before and after preprocessing are also included to show how the input standardization affected the digit representations.

References

- Deng, L., 2012. The mnist database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine* 29, 141–142.
- LeCun, Y., Bottou, L., Bengio, Y., Haffner, P., 1998. Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86, 2278–2324.
- Lin, M., Chen, Q., Yan, S., 2014. Network in network, in: *International Conference on Learning Representations*.
- Nair, A., 2022. Grid Search VS Random Search VS Bayesian Optimization | Towards Data Science — towardsdatascience.com. <https://towardsdatascience.com/grid-search-vs-random-search-vs-bayesian-optimization-2e68f57c3c46/>. [Accessed 06-05-2026].
- Otsu, N., 1979. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics* 9, 62–66.
- Semerci, Y.C., Goyal, G., 2026. Iivp2026-challenge. <https://www.kaggle.com/competitions/iivp-2026-challenge/>. Accessed: 2026-04-10.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R., 2014. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research* 15, 1929–1958.
- Watanabe, S., 2025. Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance *arXiv:2304.11127*.

**Figure 1:** Optimisation History Plot

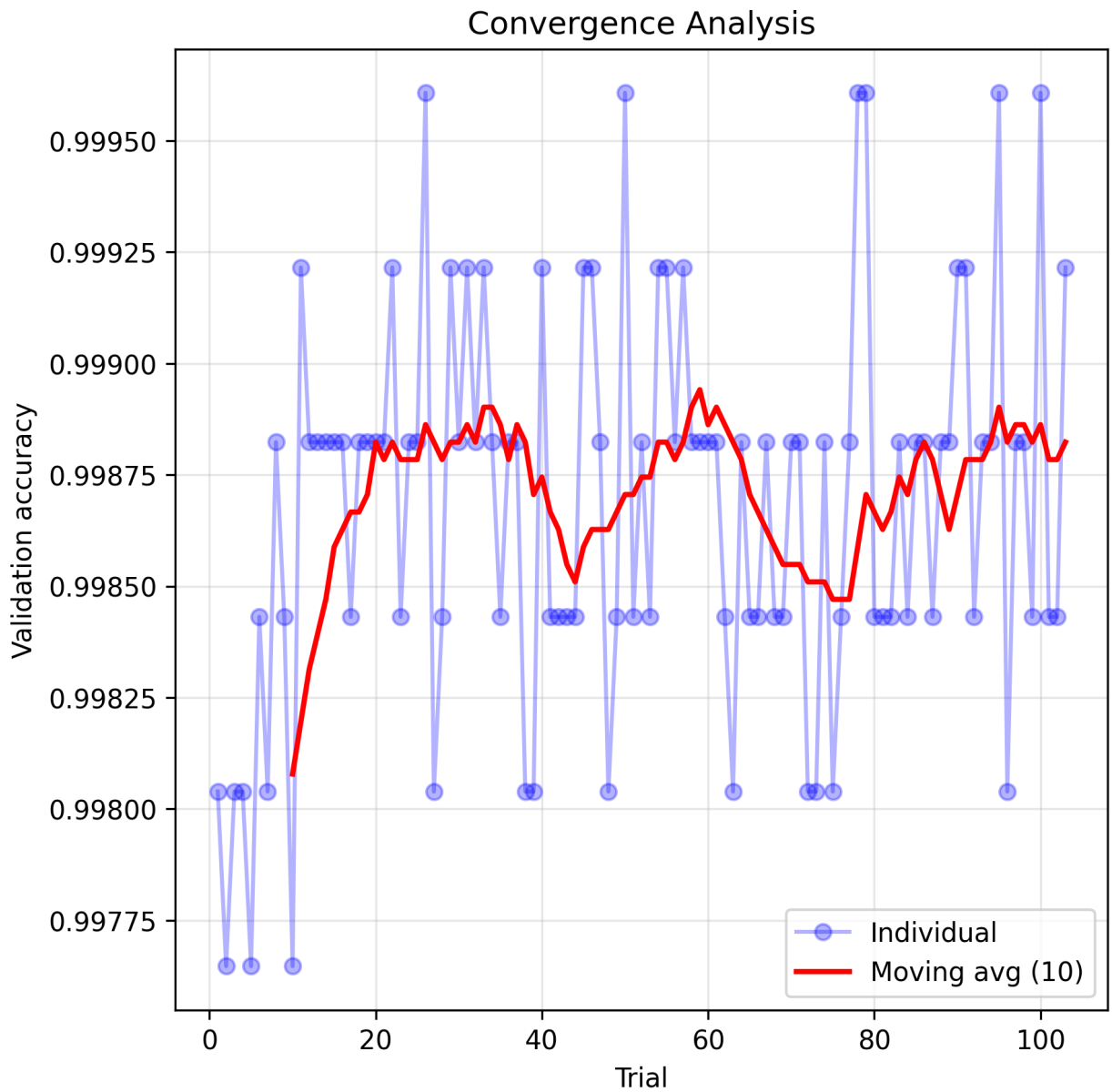


Figure 2: Convergence Analysis of Optuna Optimisation

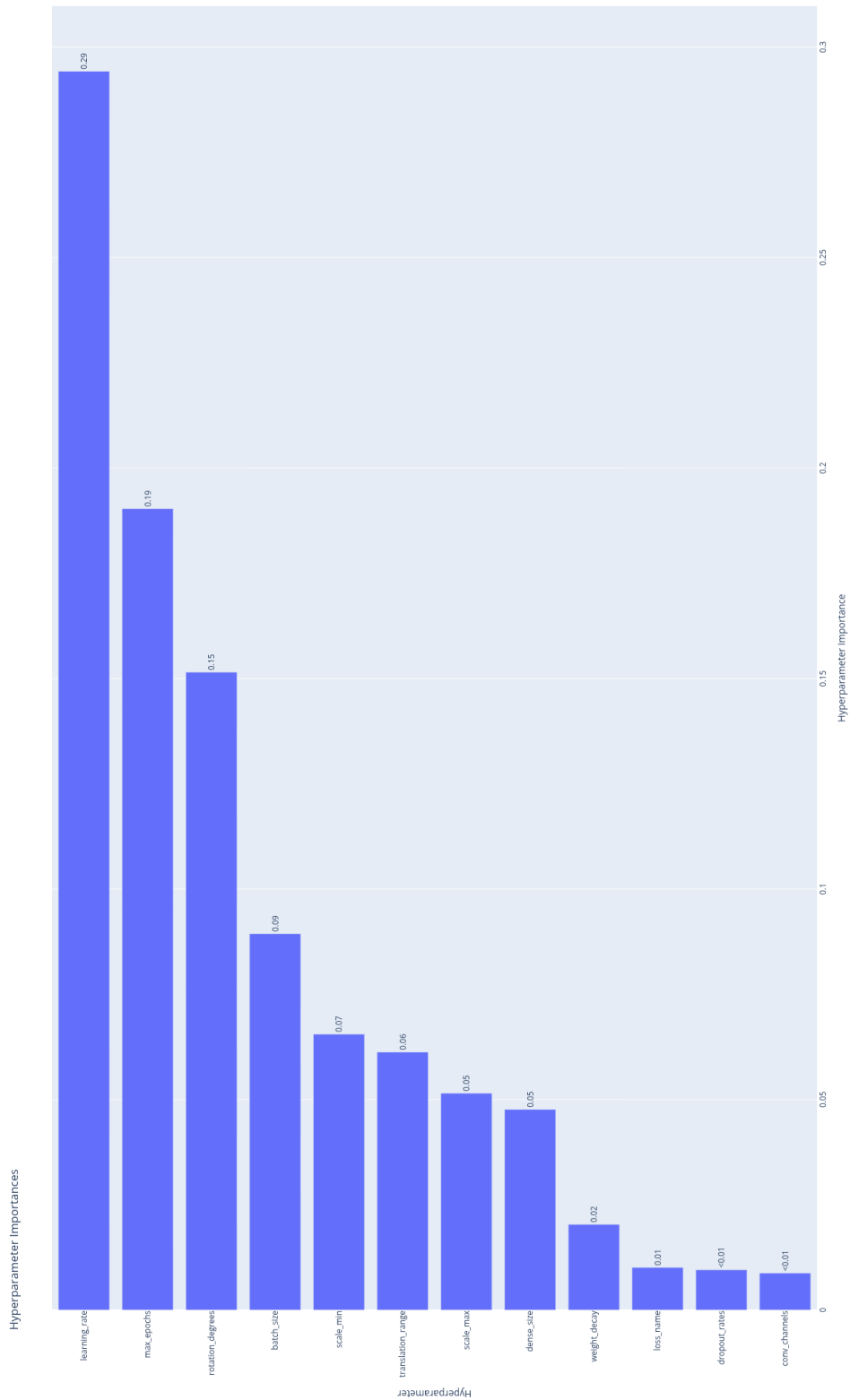


Figure 3: Hyperparameter Importance Ranked by fANOVA-Based Estimator

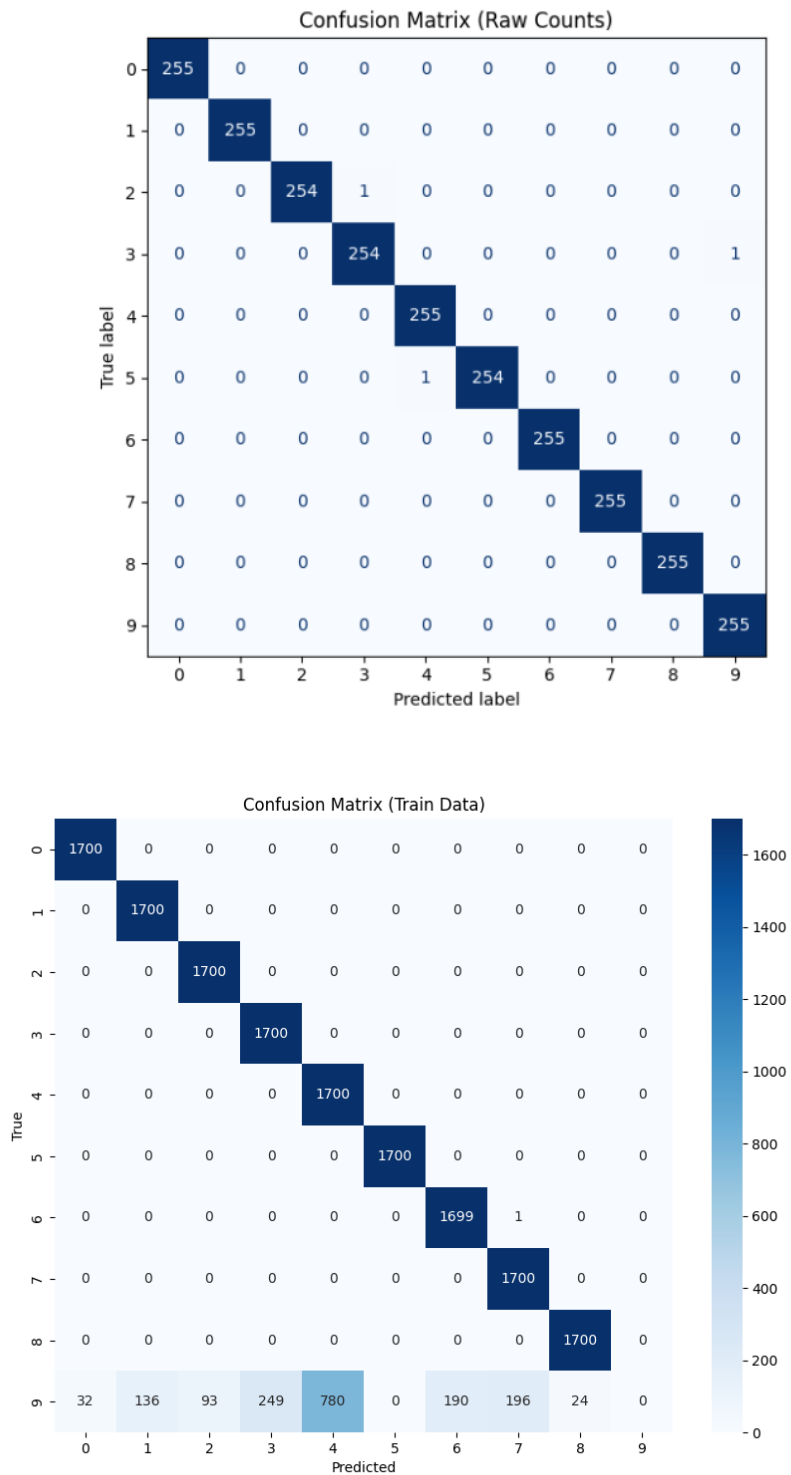


Figure 4: Confusion matrices computed on the training set. The upper matrix shows the performance of the selected CNN model, while the lower matrix shows the performance of the alternative CNN model. As you can see the two models have been tested on a different number of instances (255 vs 1700 per class), although that was not very scientific of us, its still a valuable comparison which illustrates how the alternative model struggled with identifying the last digit.



Figure 5: Examples of preprocessed digit images used as input to the selected CNN model.